

Home Assignment:

Agent used: Github copilot

1. Prepare a test strategy document based on the domain, using a chain-of-thought approach (refer to the document provided in the ZIP file)

Solution: Chain of Thought Prompt Instructions -Generate test strategy document for flight booking with Airindia -ASK me 5 questions one by one for document creation -Ask me the next question after me providing the answer -Framework should be Entrperise following the industry standard Step by step implementation

Context Act like a Airline Domain Assistant expertise in functional testing

Tone: [STRICTLY] Professional tone in review comments and summary

Output: Display the results in table format

Persona Act as a QA Manager to review the test strategy document and give the review comments

Understood. I will act as an Airline Domain Assistant with expertise in functional testing, and as a QA Manager reviewing the test strategy. We will begin by gathering requirements via 5 specific questions, one at a time, to construct an industry-standard, enterprise-level test strategy document for flight booking with Air India. I will only ask the next question after you answer the previous one.

Let's begin.

Question 1: What is the primary business objective and scope for the Air India flight booking system to be covered in this test strategy? Please specify if this scope includes only web, mobile, both, and whether it includes payment, seat selection, multi-city booking, etc.

Step Description Your Response 1 Business Objectives & Scope (TO BE FILLED BY YOU) Please provide your answer to Question 1.

You said: Business Objectives & Scope: Domestic flight booking via Business Objectives & Scope: Domestic flight booking via both web and mobile and it should include payment, seat selection, multi-city booking, etc. Copilot said: Thank you for your response. Moving on Thank you for your response. Moving on to the next question.

Question 2: What are the critical integrations and dependencies for the Air India flight booking system (e.g., payment gateways, airline inventory systems, external APIs, loyalty programs, notifications like SMS/email)? Please list all third-party and internal system touchpoints that must be validated.

Step Description Your Response 1 Business Objectives & Scope Domestic flight booking via web and mobile, including payment, seat selection, multi-city. 2 Integrations & Dependencies (TO BE FILLED BY YOU) Please provide your answer to Question 2.

You said: Third party and internal system touch points Third party and internal system touch points include Loyalty programs, sms & email notifications, payment gateways Copilot said: Thank you for your input. Let's proceed Thank you for your input. Let's proceed to the next question.

Question 3: What are the major types of testing to be included in this strategy (e.g., functional, integration, regression, user acceptance, performance, security, accessibility, localization)? Are there any compliance or regulatory requirements (such as DGCA, PCI DSS for payments, or GDPR)?

Step Description Your Response 1 Business Objectives & Scope Domestic flight booking via web and mobile, including payment, seat selection, multi-city. 2 Integrations & Dependencies Loyalty programs, sms & email notifications, payment gateways 3 Testing Types & Compliance (TO BE FILLED BY YOU) Please provide your answer to Question 3.

You said: Functional and regression testing with no compliance Functional and regression testing with no compliance requirements Copilot said: Thank you for clarifying the testing types Thank you for clarifying the testing types and compliance needs. Let's continue.

Question 4: What is the expected test data approach—will you use anonymized production data, synthetic/generated data, or require specific scenarios (e.g., frequent flyer, child, senior, group booking)? Do you need data masking or privacy considerations?

Step Description Your Response 1 Business Objectives & Scope Domestic flight booking via web and mobile, including payment, seat selection, multi-city. 2 Integrations & Dependencies Loyalty programs, sms & email notifications, payment gateways 3 Testing Types & Compliance Functional and regression testing, no compliance requirements 4 Test Data Approach (TO BE FILLED BY YOU) Please provide your answer to Question 4.

You said: Require specific scenarios e.g. frequent flyer, child, Require specific scenarios e.g. frequent flyer, child, senior, group booking. No data masking is required Copilot said: Thank you for specifying the test data Thank you for specifying the test data approach. Here is the final question.

Question 5: What is the preferred test environment and release cadence? (e.g., dedicated QA environment, use of production-like data, frequency of releases such as weekly/bi-weekly, and desired test automation levels or CI/CD integration?)

Step Description Your Response 1 Business Objectives & Scope Domestic flight booking via web and mobile, including payment, seat selection, multi-city. 2 Integrations & Dependencies Loyalty programs, sms & email notifications, payment gateways 3 Testing Types & Compliance Functional and regression testing, no compliance requirements 4 Test Data Approach Require specific scenarios e.g. frequent flyer, child, senior, group booking. No masking needed. 5 Test Environment & Release Cadence (TO BE FILLED BY YOU) Please provide your answer to Question 5.

You said: QA env and bi weekly release and QA env and bi weekly release and test automation and CI/CD is also required Copilot said: Thank you for providing all the required Thank you for providing all the required details. Below is the consolidated test strategy table for the Air India flight booking system, followed by a professional QA Manager's review and comments.

Test Strategy Document Summary Table Step Description Your Response 1 Business Objectives & Scope Domestic flight booking via web and mobile, including payment, seat selection, multi-city. 2 Integrations & Dependencies Loyalty programs, SMS & email notifications, payment gateways. 3 Testing Types & Compliance Functional and regression testing, no compliance requirements. 4 Test Data Approach Require specific scenarios: frequent flyer, child, senior, group booking. No masking needed. 5 Test Environment & Release Cadence QA environment, bi-weekly release, test automation, CI/CD pipeline required. QA Manager Review Comments Area Review Comments Scope & Objective Alignment The scope is well defined for an

enterprise product, covering core booking flows and value-added services. Integration Coverage Major touchpoints are covered. Ensure all downstream and upstream handshake points are explicitly mapped for traceability. Test Types & Risk Limiting testing types to functional and regression is acceptable given the current scope. Recommend evaluating impact if business rules or risk change. Test Data Specific test scenarios are appropriate. Recommend considering corner cases (edge date travel, infants/disabled, etc.) in data design. No masking noted—review periodically as policy may change. Environments & Automation Dedicated QA and automation with CI/CD reflect enterprise best practices. Release cadence is realistic. Ensure environment parity with production to prevent missed defects. Summary The overall test strategy aligns with enterprise and industry standards for airline booking flows. Focus on continuous improvement: Regularly revisit integration points, regulatory changes, and test automation scope as the business evolves. Ensure regression coverage expands as system complexity grows. Automation regression packs should be prioritized. Keep stakeholder communication clear, especially for dependencies and data requirements, to avoid delays during sprints and releases. If you need the step-by-step implementation or a detailed expansion of each strategy section, please let me know!

2. Design the automation framework structure using a chain-of-thought approach

Output: Analyze and review the generated results

Solution: Chain of Thought Prompt Instructions: -Design the Playwright - Cucumber - Typescript automation framework structure -ASK me 3 questions one by one for Framework creation -Ask me the next question after me providing the answer -Framework should be Production level following the industry standard Step by step implementation

Context Act like a QA Lead expertise in functional testing

Tone: [STRICTLY] Professional tone in review comments and summary

Output: Display the results in Hierarchal format

Persona Act as a Senior QA Lead to review the structure

Response: Certainly. As a Senior QA Lead specialized in functional automation, I will guide you step by step to design a production-grade Playwright-Cucumber-TypeScript test automation framework, adhering strictly to industry best practices.

Step 1: Requirement Gathering

Question 1: What is your preferred project structure organization for this framework?

Monorepo containing all test-related code and assets? Or do you require a multi-repo organization, where test automation is managed separately from the application code? Please specify your preference so we tailor scalability, CI/CD, and code management accordingly. (Please respond, and I will proceed to the next question.)

You said: I prefer Monorepo containing all test-related code I prefer Monorepo containing all test-related code and assets Copilot said: Thank you for clarifying your preference for Thank you for clarifying your preference for a monorepo structure containing all test-related code and assets. This aligns well with streamlined code sharing, unified pipeline management, and improved traceability.

Step 2: Test Scope and Environment Management

Question 2: What environments and browsers do you need to support in your production-level automation framework?

List environments (e.g., dev, QA, staging, production) Specify browsers (e.g., Chrome, Firefox, WebKit, Edge) Are there any OS considerations (e.g., Windows, Linux, MacOS) for cross-platform execution? Your answer will allow us to architect robust environment configurations and CI strategies. (Awaiting your response before proceeding to Question 3.)

You said: Environments - dev, QA, staging, production Browsers - Chrome, Firefox, WebKit, Edge OS considerations - Windows & MacOS Copilot said: Acknowledged. We will ensure the framework supports: - Acknowledged. We will ensure the framework supports:

Environments: dev, QA, staging, production Browsers: Chrome, Firefox, WebKit, Edge Operating Systems: Windows & macOS (cross-platform compatibility is non-negotiable for CI) Step 3: Reporting, Test Data, and Integration

Question 3: What are your requirements for:

Test Reporting & Dashboard integrations (e.g., Allure, HTML reports, dashboarding tools, Slack/MS Teams notifications)? Test data management (e.g., static test data files, dynamic/mock data, external data sources like DB/API)? External integrations (e.g., defect tracking/Jira, TestRail/XRay, notification channels)? Please specify so the framework incorporates the correct plugins, adapters, and data strategies. (Provide your answer to proceed to the implementation outline.)

You said: 1. Test Reporting & Dashboard integrations -

1. Test Reporting & Dashboard integrations - Allure reporting
2. Test data management - static test data files & mock data
3. External integrations - Jira Copilot said: Thank you for detailing your requirements. Here is a production-level, industry-standard framework structure design based on your inputs:

Playwright-Cucumber-TypeScript Automation Framework Structure

1. Project Root (Monorepo) Code / | — automation/ | | — config/ | | — features/ | | — src/ | | — test-data/ | | — reports/ | | — utils/ | | — .env | | — cucumber.js | | — playwright.config.ts | | — tsconfig.json | | — package.json | | — README.md | — / | — .github/ (or/.gitlab, .azure-pipelines - for CI) | — ...
2. Subfolder Structure (Automation) automation/

config/ Environment-specific config files (dev, QA, staging, production) Browser & OS configs Secure storage for secrets using .env features/ Gherkin feature files (*.feature) Step definitions mapping to feature steps Hooks: before/after scenario/feature src/ Page Object Model (POM): reusable page classes Component libraries: custom utilities, commands, API wrappers test-data/ Static test data in JSON/CSV/YAML Mock data for scenario simulation reports/ Allure HTML/XML reports (auto-generated via CI) utils/ Utility functions: logger, file reader, environment switcher, etc. 3. Configuration Files .env: Sensitive configs (per env) cucumber.js: Cucumber + Allure integration config playwright.config.ts: Playwright multi-browser + environment settings tsconfig.json: TypeScript compiler options package.json: Scripts, dependencies, Allure & Jira plugins README.md: Setup, usage, contribution guidelines 4. Integrations

Allure Reporting Step hooks for attaching screenshots, logs, and displaying cucumber results Test Data Management Import static data directly into step definitions Mock server utility for API or data simulation in utils/ Jira Integration CI hooks for auto-updating Jira tickets (possible via plugin, via API in hooks) Mapping cucumber scenario tags to Jira IDs (e.g., @JIRA-123) 5. CI/CD Considerations Cross-platform runners: Windows & macOS Matrix builds for browsers/environments Artifacts: Allure reports, logs, screenshots Notifications/integration for critical failures Implementation Flow (Step-by-Step) Initialize Repo: npm init & setup TypeScript, Playwright, Cucumber, Allure dependencies. Scaffold Structure: Create automation/ with subfolders as illustrated above. Page Object Model Setup: Define common pages and elements under src/. Gherkin Features & Steps: Author .feature files in features/, with accompanying step definitions using Playwright API. Config Environment & Browsers: Use playwright.config.ts for multiple browser/OS launching. Store env configs in config/ and use them via dotenv. Test Data Strategy: Place static data in test-data/; provide mock data via utility or inline. Allure Reporting Integration: Add Allure as cucumber formatter in cucumber.js. Configure hooks for screenshot/log attachment. Jira Integration: Apply Jira tags in feature files. Use a Jira plugin or REST API integration (typically post-results in after-hooks or via CI). CI/CD Pipeline: Configure jobs for Windows and macOS, parameterized on browser/env. Collect and publish Allure reports as build artifacts. Documentation: Document structure, conventions, pre-requisites, and contribution standards in README.md. Hierarchical Summary Code

```

├── automation
│   ├── config
│   ├── features
│   │   ├── *.feature
│   │   ├── step-definitions/
│   │   ├── src
│   │   │   ├── pages/
│   │   │   ├── test-data
│   │   │   │   ├── *.json|csv|yaml
│   │   │   │   └── mock-data/
│   │   │   ├── reports
│   │   │   │   ├── allure-results/
│   │   │   │   ├── allure-report/
│   │   │   │   └── utils
│   │   │   │       ├── logger, data-reader, environment-helpers
│   │   │   │       ├── .env
│   │   │   │       ├── cucumber.js
│   │   │   │       ├── playwright.config.ts
│   │   │   │       ├── tsconfig.json
│   │   │   │       ├── package.json
│   │   │   │       └── README.md
└── Framework Review Comments (Strictly Professional):

```

✅ Monorepo ensures central management—optimal for traceability and unified CI/CD. ✅ Supports cross-browser/OS testing; suitable for production-grade requirements. ✅ Allure integration provides comprehensive reporting for all stakeholders. ✅ Static and mock data approach is robust for functional/regression scope. ✅ Jira integration ensures traceability between test cases and business requirements. ✅ Folder-level isolation makes maintenance and scaling feasible. ❌ Any direct dependency on hard-coded config or absence of modularity will be flagged as critical debt. ❌ Ensure all secure items (secrets, API tokens) are stored outside VCS and injected securely via CI. Summary: This structure is compliant with current industry standards for enterprise-scale functional test automation. All project requirements are addressed for scalability, maintainability, traceability, and DevOps friendliness. Post-setup, focus on enforcing code review, test data reusability, and prompt reporting for long-term quality assurance.

If you require a sample package.json, playwright.config.ts, or example hook files, let me know.

You said: Yes, please provide some small sample code Yes, please provide some small sample code snippets Copilot said: Certainly. Below are concise, production-ready code snippets Certainly. Below are concise, production-ready code snippets for key components in the Playwright-Cucumber-TypeScript automation framework, as per industry standards:

```

1. Playwright Config playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
    { name: 'edge', use: { channel: 'msedge' } },
  ],
  testDir: './features',
  reporter: [['line'], ['allure-playwright']],
  use: {
    headless: true,
    baseURL: process.env.BASE_URL,
    trace: 'retain-on-failure',
    screenshot: 'only-on-failure'
  }
});

```

2. Cucumber.js Config cucumber.js module.exports = { default: ['--require-module ts-node/register', '--require ./features/step-definitions/**/*.ts', '--format @cucumber/pretty-formatter', '--format node_modules/@cucumber/allure-formatter', '--publish-quiet'].join(' ') };

3. Page Object Model Example

```
src/pages/LoginPage.ts import { Page } from '@playwright/test';
```

```
export class LoginPage { constructor(private page: Page) {}
```

```
async goto() { await this.page.goto('/login'); }
```

```
async login(username: string, password: string) { await this.page.fill('#username', username); await this.page.fill('#password', password); await this.page.click('button[type="submit"]'); }
```

4. Step Definition Example features/step-definitions/login.steps.ts import { Given, When, Then } from '@cucumber/cucumber'; import { expect } from '@playwright/test'; import { LoginPage } from '../src/pages/LoginPage';

```
Given('the user is on the login page', async function () { this.page = await this.browser.newPage(); this.loginPage = new LoginPage(this.page); await this.loginPage.goto(); });
```

```
When('the user logs in with username {string} and password {string}', async function (username: string, password: string) { await this.loginPage.login(username, password); });
```

```
Then('the dashboard should be displayed', async function () { await expect(this.page).toHaveURL(/.*dashboard/); });
```

5. Allure Hook Example (Attachment on Failure) features/support/hooks.ts import { After, Status } from '@cucumber/cucumber';

```
After(async function (scenario) { if (scenario.result?.status === Status.FAILED) { const screenshot = await this.page.screenshot(); await this.attach(screenshot, 'image/png'); } });
```

6. Sample Static Test Data (JSON) test-data/users.json [{ "username": "user1", "password": "PasswOrd!" }, { "username": "user2", "password": "Test@123" }]

7. .env Example (Environment-specific) Code BASE_URL=https://dev.example.com
JIRA_API_TOKEN=your-jira-token

8. Jira Tag in Feature File features/login.feature @smoke @JIRA-123 Feature: Login

Scenario: Successful login Given the user is on the login page When the user logs in with username "user1" and password "PasswOrd!" Then the dashboard should be displayed

Strict Review Note All code must be type-safe and reusable. Centralize selectors; avoid magic strings in steps. Never commit .env to VCS; only use .env.example for template. Always validate test data structure on load. These snippets will bootstrap your automation framework to be robust, maintainable, and CI/CD-ready. For any additional integrations (Jira API hook, advanced reporting), request a targeted snippet.